

前端工程化面试题库（30题）

1. 什么是前端工程化？它解决了哪些问题？

答案：

前端工程化是指使用工程化的方法和工具来规范前端开发流程，提高开发效率和代码质量。它主要解决以下问题：

- **代码规范化**：统一代码风格和编写规范
- **模块化管理**：解决代码复用和依赖管理问题
- **自动化构建**：自动化打包、压缩、编译等重复性工作
- **质量保障**：通过测试、代码检查等手段保证代码质量
- **部署优化**：自动化部署和版本管理
- **性能优化**：代码分割、懒加载、资源优化等

2. 请解释什么是模块化，并比较CommonJS、AMD、ES6 Module的区别？

答案：

模块化是将复杂程序按照功能拆分成独立模块的设计思想。

CommonJS (Node.js) :

- 同步加载，适用于服务端
- 使用 `require()` 导入，`module.exports` 导出
- 运行时加载，加载的是对象

AMD (RequireJS) :

- 异步加载，适用于浏览器
- 使用 `define()` 定义模块，`require()` 加载

- 依赖前置，预先加载依赖

ES6 Module:

- 编译时确定依赖关系
- 使用 `import/export` 语法
- 静态分析，支持 Tree Shaking
- 异步加载，但语法是同步的

3. Webpack的核心概念有哪些？请详细解释。

答案：

Webpack的五个核心概念：

Entry（入口）：

- 指示Webpack应该使用哪个模块作为构建依赖图的开始

Output（输出）：

- 告诉Webpack在哪里输出它所创建的bundles

Loader（加载器）：

- 让Webpack能够处理非JavaScript文件
- 如：css-loader、babel-loader、file-loader等

Plugin（插件）：

- 执行范围更广的任务，如打包优化、资源管理等
- 如：HtmlWebpackPlugin、CleanWebpackPlugin等

Mode（模式）：

- development、production、none
- 不同模式会启用相应的内置优化

4. 什么是Tree Shaking? 它的原理是什么?

答案:

Tree Shaking是一种通过静态分析来移除JavaScript上下文中未引用代码的技术。

原理:

- 基于ES6模块的静态结构特性
- 在编译时确定模块的导入导出关系
- 标记未使用的导出，在压缩阶段删除

实现条件:

- 使用ES6模块语法
- 确保代码没有副作用（side effects）
- 使用支持Tree Shaking的打包工具
- 在生产模式下启用

配置示例:

代码块

```
1  // webpack.config.js
2  module.exports = {
3    mode: 'production',
4    optimization: {
5      usedExports: true,
6      sideEffects: false
7    }
8  }
```

5. 解释代码分割（Code Splitting）的概念和实现方式?

答案:

代码分割是将代码分离到不同的bundle中，然后可以按需加载或并行加载这些文件。

实现方式：

1. 入口起点分割：

代码块

```
1  module.exports = {
2    entry: {
3      index: './src/index.js',
4      another: './src/another-module.js'
5    }
6  }
```

2. 防止重复（SplitChunksPlugin）：

代码块

```
1  optimization: {
2    splitChunks: {
3      chunks: 'all'
4    }
5  }
```

3. 动态导入：

代码块

```
1  // 异步加载模块
2  import('./math.js').then(math => {
3    console.log(math.add(16, 26));
4  });
5
6  // React.lazy
7  const LazyComponent = React.lazy(() => import('./LazyComponent'));
```

6. 什么是Babel？它的工作原理是什么？

答案：

Babel是一个JavaScript编译器，主要用于将ES6+代码转换为向后兼容的JavaScript语法。

工作原理（三个阶段）：

1. 解析（Parse）：

- 将代码字符串解析成抽象语法树（AST）

2. 转换（Transform）：

- 对AST进行遍历，在此过程中对节点进行添加、更新及移除等操作

3. 生成（Generate）：

- 将经过转换的AST再转换成代码字符串

核心组件：

- @babel/core：核心编译器
- @babel/preset-env：智能预设
- @babel/polyfill：补丁库
- 各种插件：具体的转换规则

7. 请解释什么是热模块替换（HMR），它是如何工作的？

答案：

热模块替换允许在运行时更新各种模块，而无需进行完整刷新。

工作原理：

1. 文件监听：

- Webpack监听文件变化

2. 编译更新：

- 重新编译发生变化的模块

3. 推送更新：

- 通过WebSocket将更新推送到浏览器

4. 模块替换：

- 浏览器接收更新，替换旧模块

配置示例：

代码块

```
1  // webpack.config.js
2  module.exports = {
3    devServer: {
4      hot: true
5    },
6    plugins: [
7      new webpack.HotModuleReplacementPlugin()
8    ]
9  }
10
11 // 代码中接受HMR
12 if (module.hot) {
13   module.hot.accept('./library.js', function() {
14     // 处理更新逻辑
15   })
16 }
```

8. 什么是Polyfill和Shim？它们有什么区别？

答案：

Polyfill:

- 用于实现浏览器并不支持的原生API的代码
- 模拟标准API的行为
- 例: Promise polyfill、Array.prototype.includes polyfill

Shim:

- 更广泛的概念, 用于修复或增强现有API
- 可能改变现有API的行为
- 不一定遵循标准规范

主要区别:

- Polyfill严格按照标准实现, Shim可能有自己的实现方式
- Polyfill只添加缺失功能, Shim可能修改现有功能
- Polyfill更注重兼容性, Shim更注重功能增强

使用示例:

代码块

```
1  // Polyfill示例
2  if (!Array.prototype.includes) {
3    Array.prototype.includes = function(searchElement) {
4      return this.indexOf(searchElement) !== -1;
5    };
6  }
```

9. 解释什么是Source Map, 它有什么作用?

答案:

Source Map是一个信息文件, 里面储存着位置信息, 用于调试压缩/编译后的代码。

作用:

- 将压缩/编译后的代码映射回原始源代码
- 便于在浏览器中调试
- 保持生产环境代码的优化，同时保证开发体验

类型：

- **eval**：每个模块使用eval()执行，并生成DataURL形式的SourceMap
- **source-map**：生成独立的.map文件
- **inline-source-map**：将SourceMap以DataURL形式嵌入
- **cheap-source-map**：不包含列信息，只有行信息
- **hidden-source-map**：生成SourceMap但不在bundle中引用

配置：

代码块

```
1  // webpack.config.js
2  module.exports = {
3    devtool: 'source-map' // 生产环境
4    // devtool: 'eval-source-map' // 开发环境
5  }
```

10. 什么是微前端？它解决了什么问题？

答案：

微前端是一种将前端应用分解为更小、更简单的能够独立开发、测试、部署的微应用的架构风格。

解决的问题：

- **技术栈无关**：不同团队可以使用不同技术栈
- **独立部署**：各个微应用可以独立发布
- **团队自治**：不同团队负责不同业务模块
- **增量升级**：可以逐步迁移遗留系统
- **容错隔离**：单个应用出错不影响整体

实现方案：

- **Single-SPA**：微前端框架
- **Module Federation**：Webpack 5 的模块联邦
- **iframe**：简单但有限制
- **Web Components**：标准化组件

挑战：

- 应用间通信
- 样式隔离
- 公共依赖管理
- 性能优化

11. 请解释Webpack的打包原理？

答案：

Webpack打包原理可以分为以下几个步骤：

1. 初始化参数：

- 合并配置文件和命令行参数

2. 开始编译：

- 初始化Compiler对象，加载所有配置的插件

3. 确定入口：

- 根据entry配置找到所有入口文件

4. 编译模块：

- 从入口文件出发，调用所有配置的Loader对模块进行翻译
- 找出该模块依赖的模块，递归本步骤

5. 完成模块编译：

- 得到每个模块被翻译后的最终内容以及依赖关系

6. 输出资源：

- 根据入口和模块间的依赖关系，组装成一个个包含多个模块的Chunk
- 再把每个Chunk转换成一个单独的文件加入到输出列表

7. 输出完成：

- 根据配置确定输出的路径和文件名，把文件内容写入到文件系统

12. 什么是ESLint？ 如何配置和使用？

答案：

ESLint是一个用于识别和报告JavaScript代码中模式匹配的工具，目标是保证代码的一致性和避免错误。

主要功能：

- 语法错误检查
- 代码风格检查
- 潜在问题检查
- 自动修复部分问题

配置方式：

1. 配置文件（.eslintrc.js）：

代码块

```
1  module.exports = {  
2    env: {  
3      browser: true,  
4      es2021: true,
```

```
5     node: true
6   },
7   extends: [
8     'eslint:recommended',
9     '@vue/typescript/recommended'
10  ],
11  parserOptions: {
12    ecmaVersion: 12,
13    sourceType: 'module'
14  },
15  rules: {
16    'no-console': 'warn',
17    'no-unused-vars': 'error'
18  }
19 }
```

2. 集成到构建工具：

代码块

```
1  // webpack.config.js
2  module.exports = {
3    module: {
4      rules: [
5        {
6          test: /\.js$/,
7          loader: 'eslint-loader',
8          enforce: 'pre'
9        }
10     ]
11   }
12 }
```

13. 什么是Prettier？它与ESLint有什么区别？

答案：

Prettier：

- 代码格式化工具

- 专注于代码风格统一
- 支持多种语言
- 配置项较少，固执己见

ESLint:

- 代码质量检查工具
- 专注于代码质量和潜在错误
- 主要针对JavaScript
- 配置灵活，规则丰富

主要区别:

- **职责不同:** Prettier负责格式化，ESLint负责代码质量
- **冲突处理:** 某些规则可能冲突，需要配置解决
- **使用场景:** 通常配合使用，各司其职

配合使用:

代码块

```
1  // .eslintrc.js
2  module.exports = {
3    extends: [
4      'eslint:recommended',
5      'prettier' // 关闭与Prettier冲突的规则
6    ],
7    plugins: ['prettier'],
8    rules: {
9      'prettier/prettier': 'error'
10   }
11 }
```

14. 什么是Monorepo? 它有什么优缺点?

答案:

Monorepo是一种项目代码管理策略，即在一个仓库中管理多个项目。

优点：

- **代码共享：**容易共享代码和依赖
- **统一工具链：**统一的构建、测试、部署流程
- **原子提交：**跨项目的更改可以在一次提交中完成
- **依赖管理：**更容易管理项目间依赖
- **重构友好：**大规模重构更容易进行

缺点：

- **仓库体积大：**随着项目增多，仓库会变得很大
- **构建时间长：**可能需要构建整个仓库
- **权限控制：**难以对不同项目设置不同权限
- **学习成本：**需要学习相应的工具链

常用工具：

- **Lerna：**JavaScript项目的Monorepo工具
- **Nx：**可扩展的开发工具
- **Rush：**Microsoft开发的Monorepo工具
- **Yarn Workspaces：**Yarn的工作空间功能

15. 解释什么是持续集成（CI）和持续部署（CD）？

答案：

持续集成（CI - Continuous Integration）：

- 开发人员频繁地将代码集成到主干
- 每次集成都通过自动化构建来验证
- 快速发现集成错误

持续部署（CD - Continuous Deployment/Delivery）：

- **Continuous Delivery**：确保代码随时可以部署到生产环境
- **Continuous Deployment**：每次通过CI的代码自动部署到生产环境

前端CI/CD流程：

1. **代码提交**：开发者推送代码到仓库
2. **自动构建**：触发构建流程
3. **代码检查**：ESLint、TypeScript检查
4. **自动测试**：单元测试、集成测试
5. **构建打包**：Webpack打包
6. **部署**：部署到测试/生产环境

常用工具：

- **Jenkins**、**GitHub Actions**、**GitLab CI**
- **Docker**、**Kubernetes**
- **AWS**、**阿里云**等云服务

16. 什么是Package.json？请解释其主要字段的作用？

答案：

Package.json是Node.js项目的配置文件，包含项目的元数据和依赖信息。

主要字段：

基本信息：

- **name**：项目名称
- **version**：版本号（遵循语义化版本）
- **description**：项目描述
- **keywords**：关键词数组
- **author**：作者信息

依赖管理：

- **dependencies**：生产环境依赖
- **devDependencies**：开发环境依赖
- **peerDependencies**：同伴依赖
- **optionalDependencies**：可选依赖

脚本和配置：

- **scripts**：可执行脚本
- **main**：入口文件
- **engines**：Node.js版本要求
- **browserslist**：浏览器兼容性配置

示例：

代码块

```
1  {
2    "name": "my-project",
3    "version": "1.0.0",
4    "scripts": {
5      "dev": "webpack serve",
6      "build": "webpack --mode=production"
7    },
8    "dependencies": {
9      "vue": "^3.0.0"
10   },
11   "devDependencies": {
12     "webpack": "^5.0.0"
13   }
14 }
```

17. 解释npm、yarn、pnpm的区别？

答案：

npm (Node Package Manager) :

- Node.js官方包管理器
- 使用node_modules扁平化结构
- 有lock文件 (package-lock.json)
- 安装速度相对较慢

yarn:

- Facebook开发的包管理器
- 并行下载，速度更快
- 更好的缓存机制
- yarn.lock文件锁定版本
- 支持工作空间 (Workspaces)

pnpm:

- 使用硬链接和符号链接
- 节省磁盘空间
- 更严格的依赖管理
- 安装速度快
- 天然支持Monorepo

主要区别:

- **存储方式:** pnpm使用全局存储，避免重复
- **安装速度:** pnpm > yarn > npm
- **磁盘占用:** pnpm最少，npm最多
- **依赖管理:** pnpm最严格，避免幽灵依赖

18. 什么是语义化版本 (Semantic Versioning) ?

答案:

语义化版本是一套版本号命名规则，格式为：主版本号.次版本号.修订号（MAJOR.MINOR.PATCH）。

版本号递增规则：

- **MAJOR**：不兼容的API修改
- **MINOR**：向下兼容的功能性新增
- **PATCH**：向下兼容的问题修正

预发布版本：

- **alpha**：内部测试版本
- **beta**：公开测试版本
- **rc**：候选发布版本

npm中的版本范围：

- **^1.2.3**：兼容1.x.x，但不包括2.0.0
- **~1.2.3**：兼容1.2.x，但不包括1.3.0
- **1.2.3**：精确版本
- **>=1.2.3**：大于等于指定版本
- **latest**：最新版本

示例：

代码块

```
1  {
2    "dependencies": {
3      "vue": "^3.2.0",      // 3.2.0 <= version < 4.0.0
4      "lodash": "~4.17.21", // 4.17.21 <= version < 4.18.0
5      "axios": "0.27.2"    // 精确版本
6    }
7  }
```

19. 什么是Vite？它相比Webpack有什么优势？

答案：

Vite是一个现代化的前端构建工具，由Vue.js作者尤雨溪开发。

核心特性：

- **极速的服务启动：**使用原生ES模块
- **轻量快速的热重载：**基于ESM的HMR
- **丰富的功能：**TypeScript、JSX、CSS等开箱即用
- **优化的构建：**使用Rollup进行生产构建

相比Webpack的优势：

开发环境：

- **启动速度快：**不需要打包，直接使用ES模块
- **热更新快：**只需要重新请求单个模块
- **配置简单：**零配置即可使用

生产环境：

- **构建速度快：**使用esbuild进行预构建
- **更好的Tree Shaking：**基于Rollup
- **现代化输出：**原生支持ES模块

适用场景：

- 新项目推荐使用Vite
- 现代浏览器环境
- Vue、React等现代框架项目

局限性：

- 生态系统相对较新
- 某些老旧插件可能不兼容

20. 解释什么是预处理器？常见的CSS预处理器有哪些？

答案：

预处理器是一种工具，它可以让你使用特殊的语法来生成CSS。

CSS预处理器的优势：

- **变量：**定义可重用的值
- **嵌套：**层级化的样式编写
- **混合（Mixin）：**可重用的样式块
- **函数：**动态生成样式
- **模块化：**@import功能增强

常见的CSS预处理器：

Sass/SCSS：

代码块

```
1  $primary-color: #333;
2  $margin: 16px;
3
4  .header {
5      color: $primary-color;
6      margin: $margin;
7
8      &:hover {
9          color: lighten($primary-color, 20%);
10     }
11 }
```

Less：

代码块

```
1  @primary-color: #333;
2  @margin: 16px;
3
```

```
4  .header {
5    color: @primary-color;
6    margin: @margin;
7
8    &:hover {
9      color: lighten(@primary-color, 20%);
10   }
11 }
```

Stylus:

代码块

```
1  primary-color = #333
2  margin = 16px
3
4  .header
5    color primary-color
6    margin margin
7
8    &:hover
9      color lighten(primary-color, 20%)
```

21. 什么是PostCSS? 它与预处理器有什么区别?

答案:

PostCSS是一个用JavaScript工具和插件转换CSS代码的工具。

PostCSS特点:

- **插件化架构**: 功能通过插件实现
- **后处理**: 处理已有的CSS
- **可定制**: 可以选择需要的功能
- **性能好**: 只处理需要的部分

与预处理器的区别:

- **处理时机**：PostCSS是后处理，预处理器是预处理
- **语法**：PostCSS使用标准CSS语法
- **功能**：PostCSS通过插件扩展，预处理器有固定语法
- **兼容性**：PostCSS可以处理现有CSS

常用插件：

- **autoprefixer**：自动添加浏览器前缀
- **cssnano**：CSS压缩优化
- **postcss-preset-env**：使用未来CSS语法
- **postcss-import**：处理@import

配置示例：

代码块

```
1  // postcss.config.js
2  module.exports = {
3    plugins: [
4      require('autoprefixer'),
5      require('cssnano')({
6        preset: 'default'
7      })
8    ]
9  }
```

22. 什么是Webpack的Loader和Plugin？请举例说明。

答案：

Loader：

- 用于转换模块的源代码
- 在import或加载模块时预处理文件
- 从右到左（或从下到上）执行

常见Loader:

代码块

```
1  module.exports = {
2    module: {
3      rules: [
4        // 处理CSS文件
5        {
6          test: /\.css$/,
7          use: ['style-loader', 'css-loader']
8        },
9        // 处理JavaScript文件
10       {
11         test: /\.js$/,
12         exclude: /node_modules/,
13         use: {
14           loader: 'babel-loader',
15           options: {
16             presets: ['@babel/preset-env']
17           }
18         }
19       },
20       // 处理图片文件
21       {
22         test: /\.(png|jpg|gif)$/,
23         use: ['file-loader']
24       }
25     ]
26   }
27 }
```

Plugin:

- 执行范围更广的任务
- 可以访问整个编译生命周期
- 通过钩子系统工作

常见Plugin:

代码块

```
1  const HtmlWebpackPlugin = require('html-webpack-plugin');
```

```
2  const CleanWebpackPlugin = require('clean-webpack-plugin');
3
4  module.exports = {
5    plugins: [
6      // 清理输出目录
7      new CleanWebpackPlugin(),
8      // 生成HTML文件
9      new HtmlWebpackPlugin({
10       template: './src/index.html'
11     }),
12     // 定义环境变量
13     new webpack.DefinePlugin({
14       'process.env.NODE_ENV': JSON.stringify('production')
15     })
16   ]
17 }
```

23. 解释什么是Bundle Splitting和Chunk?

答案:

Bundle:

- Webpack打包后的文件
- 包含多个模块的代码集合
- 最终输出到dist目录的文件

Chunk:

- Webpack内部用来管理打包过程的代码块
- 一个Chunk可能包含多个模块
- 最终会生成一个或多个Bundle

Bundle Splitting策略:

1. 入口分割:

代码块

```
1 module.exports = {
2   entry: {
3     app: './src/app.js',
4     vendor: './src/vendor.js'
5   }
6 }
```

2. SplitChunksPlugin:

代码块

```
1 optimization: {
2   splitChunks: {
3     chunks: 'all',
4     cacheGroups: {
5       vendor: {
6         test: /[\\/]node_modules[\\/]/,
7         name: 'vendors',
8         chunks: 'all'
9       },
10      common: {
11        minChunks: 2,
12        chunks: 'all',
13        name: 'common'
14      }
15    }
16  }
17 }
```

3. 动态导入:

代码块

```
1 // 创建新的Chunk
2 import('./lazy-module.js').then(module => {
3   // 使用模块
4 });
```

优势:

- 减少初始加载时间
- 更好的缓存策略
- 按需加载

24. 什么是PWA？它包含哪些技术？

答案：

PWA（Progressive Web App）是一种使用现代Web技术构建的应用程序，提供类似原生应用的用户体验。

核心技术：

1. Service Worker：

- 在后台运行的脚本
- 提供离线功能
- 拦截网络请求
- 推送通知

2. Web App Manifest：

- JSON文件，定义应用元数据
- 支持添加到主屏幕
- 定义启动画面、图标等

3. HTTPS：

- 安全连接要求
- Service Worker的前提条件

实现示例：

manifest.json：

代码块

```
2   "name": "My PWA App",
3   "short_name": "PWA App",
4   "start_url": "/",
5   "display": "standalone",
6   "background_color": "#ffffff",
7   "theme_color": "#000000",
8   "icons": [
9     {
10      "src": "/icon-192.png",
11      "sizes": "192x192",
12      "type": "image/png"
13    }
14  ]
15 }
```

Service Worker:

代码块

```
1  // 缓存策略
2  self.addEventListener('fetch', event => {
3    event.respondWith(
4      caches.match(event.request)
5        .then(response => response || fetch(event.request))
6    );
7  });
```

25. 什么是Serverless？它对前端开发有什么影响？

答案：

Serverless是一种云计算执行模型，开发者无需管理服务器基础设施。

特点：

- **无服务器管理**：云服务商管理服务器
- **按需付费**：只为实际使用付费
- **自动扩缩容**：根据负载自动调整

- **事件驱动**：通过事件触发执行

对前端的影响：

1. JAMstack架构：

- JavaScript + APIs + Markup
- 静态站点生成
- 通过API调用后端服务

2. 边缘计算：

- CDN边缘节点执行代码
- 降低延迟
- 提升用户体验

3. 全栈开发：

- 前端开发者可以编写后端逻辑
- 简化部署流程
- 降低运维成本

常见平台：

- **Vercel**：专注于前端部署
- **Netlify**：静态站点托管
- **AWS Lambda**：函数即服务
- **Cloudflare Workers**：边缘计算

26. 解释什么是Micro Frontends的实现方案？

答案：

微前端的实现方案有多种，每种都有其适用场景：

1. 构建时集成：

代码块

```
1  // 将微应用作为npm包发布
2  import MicroApp from '@company/micro-app';
3
4  function App() {
5    return (
6      <div>
7        <MicroApp />
8      </div>
9    );
10 }
```

2. 运行时集成 - Single-SPA:

代码块

```
1  // 注册微应用
2  registerApplication({
3    name: 'vue-app',
4    app: () => System.import('@company/vue-app'),
5    activeWhen: '/vue'
6  });
7
8  // 启动Single-SPA
9  start();
```

3. Module Federation:

代码块

```
1  // webpack.config.js - 主应用
2  new ModuleFederationPlugin({
3    name: 'shell',
4    remotes: {
5      mfApp: 'mfApp@http://localhost:3001/remoteEntry.js'
6    }
7  });
8
9  // 使用远程模块
```

```
10  const RemoteComponent = React.lazy(() => import('mfApp/Component'));
```

4. Web Components:

代码块

```
1  // 定义微前端组件
2  class MicroFrontend extends HTMLElement {
3    connectedCallback() {
4      this.innerHTML = '<div>Micro Frontend Content</div>';
5    }
6  }
7
8  customElements.define('micro-frontend', MicroFrontend);
```

5. iframe方案:

代码块

```
1  <iframe
2    src="http://micro-app.com"
3    sandbox="allow-scripts allow-same-origin">
4  </iframe>
```

27. 什么是性能预算（Performance Budget）？ 如何实施？

答案:

性能预算是为网站或应用设定的性能指标限制，用于确保用户体验不会因为功能增加而降低。

常见指标:

- **包大小:** JavaScript、CSS文件大小
- **加载时间:** 首屏加载时间、完全加载时间
- **网络请求:** HTTP请求数量
- **Core Web Vitals:** LCP、FID、CLS等

实施方法：

1. Webpack Bundle Analyzer:

代码块

```
1  // webpack.config.js
2  const BundleAnalyzerPlugin = require('webpack-bundle-
  analyzer').BundleAnalyzerPlugin;
3
4  module.exports = {
5    plugins: [
6      new BundleAnalyzerPlugin({
7        analyzerMode: 'static',
8        openAnalyzer: false
9      })
10   ]
11 }
```

2. 大小限制:

代码块

```
1  // webpack.config.js
2  module.exports = {
3    performance: {
4      maxAssetSize: 250000,      // 250kb
5      maxEntrypointSize: 250000, // 250kb
6      hints: 'error'
7    }
8  }
```

3. CI/CD集成:

代码块

```
1  # GitHub Actions
2  - name: Check bundle size
3    run: |
4      npm run build
```

4. 监控工具：

- **Lighthouse CI**：自动化性能测试
- **WebPageTest**：性能分析
- **Bundle Size Bot**：PR中显示包大小变化

28. 什么是Design System？它在前端工程化中的作用是什么？

答案：

Design System是一套完整的设计标准、组件库和工具，用于创建一致的用户体验。

组成部分：

- **设计原则**：颜色、字体、间距等基础规范
- **组件库**：可复用的UI组件
- **模式库**：常见的交互模式
- **工具链**：开发、测试、文档工具

在前端工程化中的作用：

1. 一致性保证：

代码块

```
1  // 统一的主题配置
2  const theme = {
3    colors: {
4      primary: '#007bff',
5      secondary: '#6c757d'
6    },
7    spacing: {
8      small: '8px',
9      medium: '16px',
10     large: '24px'
```

```
11    }  
12 }
```

2. 开发效率提升：

代码块

```
1  // 使用设计系统组件  
2  import { Button, Card, Input } from '@company/design-system';  
3  
4  function LoginForm() {  
5    return (  
6      <Card>  
7        <Input placeholder="Username" />  
8        <Button variant="primary">Login</Button>  
9      </Card>  
10    );  
11  }
```

3. 维护性改善：

- 集中管理样式和组件
- 统一的更新和修复
- 版本控制和文档

4. 团队协作：

- 设计师和开发者的共同语言
- 减少沟通成本
- 提高交付质量

实现工具：

- **Storybook**：组件开发和文档
- **Figma**：设计协作
- **Styled System**：主题化样式系统

29. 解释什么是Headless CMS？它对前端开发的意义是什么？

答案：

Headless CMS是一种后端内容管理系统，只提供内容管理功能，不包含前端展示层。

特点：

- **API优先**：通过API提供内容
- **前后端分离**：前端可以自由选择技术栈
- **多渠道发布**：同一内容可以发布到多个平台
- **开发者友好**：更灵活的开发方式

对前端开发的意义：

1. 技术栈自由：

代码块

```
1  // 可以使用任何前端框架
2  // React + Headless CMS
3  useEffect(() => {
4    fetch('/api/content')
5      .then(res => res.json())
6      .then(data => setContent(data));
7  }, []);
```

2. 性能优化：

- 静态站点生成（SSG）
- 服务端渲染（SSR）
- 边缘缓存

3. JAMstack架构：

代码块

```
1  // Next.js + Headless CMS
```

```
2 export async function getStaticProps() {
3   const posts = await cms.getPosts();
4
5   return {
6     props: { posts },
7     revalidate: 60 // ISR
8   };
9 }
```

常见Headless CMS:

- **Strapi**: 开源Node.js CMS
- **Contentful**: 云端CMS服务
- **Sanity**: 实时协作CMS
- **Ghost**: 专注于博客的CMS

30. 什么是Web Assembly (WASM) ? 它对前端性能优化有什么帮助?

答案:

WebAssembly是一种可以在现代Web浏览器中运行的新型代码格式，提供接近原生的性能。

特点:

- **高性能**: 接近原生代码的执行速度
- **安全**: 在沙盒环境中运行
- **跨平台**: 支持多种编程语言编译
- **与JavaScript互操作**: 可以与JS代码协同工作

性能优化帮助:

1. 计算密集型任务:

代码块

```
1 // 加载WASM模块
2 WebAssembly.instantiateStreaming(fetch('math.wasm'))
```

```
3     .then(result => {
4         const { calculate } = result.instance.exports;
5
6         // 使用WASM函数进行复杂计算
7         const result = calculate(largeDataSet);
8     });
```

2. 图像/视频处理：

代码块

```
1 // 使用WASM进行图像处理
2 const processImage = async (imageData) => {
3     const wasmModule = await loadWasmModule();
4     return wasmModule.processImage(imageData);
5 };
```

3. 游戏引擎：

- 将C++游戏引擎编译为WASM
- 在浏览器中运行高性能游戏

4. 科学计算：

- 数据分析和可视化
- 机器学习模型推理

编译工具：

- **Emscripten**：C/C++到WASM
- **AssemblyScript**：TypeScript-like语法
- **Rust**：原生支持WASM编译
- **Go**：支持WASM目标

使用场景：

- 需要高性能计算的应用

- 现有C/C++代码的Web移植
- 实时音视频处理
- 加密算法实现